In [1]:
```
1 + 1 #Addition
```
Out[1]: 2

In [2]:
```
2-1
```
Out[2]: 1

In [3]:
```
3 * 4
```
Out[3]: 12

In [4]:
```
8 / 4 #Division
```
Out[4]: 2.0

In [5]:
```
8/5 #float division
```
Out[5]: 1.6

In [6]:
```
8/4 #float division
```
Out[6]: 2.0

In [7]:
```
8//4 #integer division
```
Out[7]: 2

In [8]:
```
8+9-7
```
Out[8]: 10

In [9]:
```
8 + 8 - #syntax error due to - symbol and not completed the equation
```

```
  Cell In[9], line 1
    8 + 8 - #syntax error due to - symbol and not completed the equation
            ^
SyntaxError: invalid syntax
```

In [ ]:
```
5 + 5 * 5
```

In [ ]:
```
(5 + 5) * 5 #BODMAS (Bracket || Oders || Divide || Multiply || Add || Substract)
```

In [ ]:
```
2 * 2 * 2 * 2 * 2 #exponentaion
```

In [ ]:
```
2 * 5 #multiplication
```

In [ ]:
```
2 ** 5 #power
```

In [ ]:
```
15 % 2 #Modulus
```

In [ ]:
```
10 % 2
```

In [ ]:
```
15 %% 2
```

In [ ]:
```
3 + 'nit' #int + string not possible
```

In [ ]:
```
a,b,c,d,e = 15, 7.8, 'nit', 8+9j, True
print(a)
```

```
print(b)
print(c)
print(d)
print(e)
```

In [ ]:
```
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
```

In [ ]:
```
'Naresh IT'
```

In [ ]:
```
print('Max it')
```

In [ ]:
```
"max it technology"
```

In [ ]:
```
s1 = 'max it technology'
s1
```

In [ ]:
```
a = 2
b = 3
a + b
```

In [ ]:
```
c = a + b
c
```

In [10]:
```
a = 3
b = 'hi'
type(b)
```

Out[10]:
```
str
```

In [11]:
```
print('max it's "Technology"') # \ has some special meaning to ignore the error
```

```
  Cell In[11], line 1
    print('max it's "Technology"') # \ has some special meaning to ignore the error
                                  ^
SyntaxError: unterminated string literal (detected at line 1)
```

In [12]:
```
print('max it\'s "Technology"') # \ has some special meaning to ignore the error
```

```
max it's "Technology"
```

In [13]:
```
print('max it', 'Technology')
```

```
max it Technology
```

In [14]:
```
print("max it', 'Technology")
```

```
max it', 'Technology
```

In [15]:
```
# print the nit 2 times
'nit' + 'nit
```

```
  Cell In[15], line 2
    'nit' + 'nit
            ^
SyntaxError: unterminated string literal (detected at line 2)
```

In [16]:
```
'nit' 'nit'
```

Out[16]:
```
'nitnit'
```

In [17]:
```
#5 time print
5 * 'nit'
```

Out[17]:
```
'nitnitnitnitnit'
```

In [18]:
```
5 * ' nit' #space between words
```

Out[18]:
```
' nit nit nit nit nit'
```

In [19]:
```
print('c:\nit') #\n refers to new line
```

```
c:
it
```

In [20]:
```
print(r'c:\nit') #r refers to raw string
```

```
c:\nit
```

In [21]:
```
2
```

Out[21]:
```
2
```

In [22]:
```
x = 2 # x is variable/identifier/object, 2 is the value
x
```

Out[22]:
```
2
```

In [23]:
```
x + 3
```

Out[23]:
```
5
```

In [24]:
```
y = 3
y
```

Out[24]:
```
3
```

In [25]:
```
x + y
```

Out[25]:
```
5
```

In [26]:
```
x = 9
x
```

Out[26]:
```
9
```

In [27]:
```
x + y
```

Out[27]:
```
12
```

In [28]:
```
x + 10
```

Out[28]:
```
19
```

In [29]:
```
_+y # _ underscore stores the previous result but in some jupyter version it doesn't work
```

Out[29]:
```
22
```

In [30]:
```
19 + y
```

Out[30]:
```
22
```

In [31]:
```
22 + y
```

Out[31]: 25

In [32]:
```
y
```

Out[32]: 3

In [33]:
```
# string variable
name = 'nit'
```

In [34]:
```
name
```

Out[34]: 'nit'

In [35]:
```
name + 'technology'
```

Out[35]: 'nittechnology'

In [36]:
```
name + ' technology' #add space in string
```

Out[36]: 'nit technology'

In [37]:
```
name 'technology'
```

```
  Cell In[37], line 1
    name 'technology'
         ^
SyntaxError: invalid syntax
```

In [38]:
```
name
```

Out[38]: 'nit'

In [39]:
```
len(name) #use len() function to find the length of the given variable
```

Out[39]: 3

In [40]:
```
name[0] #pythonn index begins with 0
```

Out[40]: 'n'

In [41]:
```
name[5]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[41], line 1
----> 1 name[5]

IndexError: string index out of range
```

In [42]:
```
name[7] #gives error because name variable has only upto 2nd index
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[42], line 1
----> 1 name[7] #gives error because name variable has only upto 2nd index

IndexError: string index out of range
```

In [43]:
```
name[-1]
```

Out[43]: 't'

In [44]:
```
name[-2]
```

Out[44]:
```
'i'
```

In [45]:
```
name[-6] #the negative index of the name variable is -3 so it gives error
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[45], line 1
----> 1 name[-6] #the negative index of the name variable is -3 so it gives error

IndexError: string index out of range
```

Slicing

In [46]:
```
name
```

Out[46]:
```
'nit'
```

In [47]:
```
name[0:1] #to print 2 characters
```

Out[47]:
```
'n'
```

In [49]:
```
name[0:2]
```

Out[49]:
```
'ni'
```

In [50]:
```
name[1:4]
```

Out[50]:
```
'it'
```

In [51]:
```
name[1:]
```

Out[51]:
```
'it'
```

In [52]:
```
name[:4]
```

Out[52]:
```
'nit'
```

In [53]:
```
name[3:9]
```

Out[53]:
```
''
```

In [54]:
```
name
```

Out[54]:
```
'nit'
```

In [55]:
```
name1 = 'fine' # change the string fine to dine
name1
```

Out[55]:
```
'fine'
```

In [56]:
```
name[0:1]
```

Out[56]:
```
'n'
```

In [57]:
```
name1[0:1] = 'd' # I want to change 1st charracter of naresh (n) - t
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[57], line 1
----> 1 name1[0:1] = 'd' # I want to change 1st charracter of naresh (n) - t

TypeError: 'str' object does not support item assignment
```

In [58]:
```
name1[0] = 'd' #string in python are immutable
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[58], line 1
----> 1 name1[0] = 'd' #string in python are immutable

TypeError: 'str' object does not support item assignment
```

In [59]:
```
name1
```

Out[59]:
```
'fine'
```

In [60]:
```
name1[1:]
```

Out[60]:
```
'ine'
```

In [61]:
```
'd' + name1[1:] #I want to change fine to dine
```

Out[61]:
```
'dine'
```

In [62]:
```
num1.insert(2,'nit') #insert the value as per index values i.e., 2nd index we are assigning n
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[62], line 1
----> 1 num1.insert(2,'nit') #insert the value as per index values i.e., 2nd index we are assi
gning nit

NameError: name 'num1' is not defined
```

introduce to ID()

In [63]:
```
#variable address
num = 5
id(num)
```

Out[63]:
```
140719961646120
```

In [64]:
```
name = 'nit'
id(name) #Address will be different for both
```

Out[64]:
```
2493758691040
```

In [65]:
```
a = 10
id(a)
```

Out[65]:
```
140719961646280
```

In [66]:
```
b = a #thats why python is more memory efficient
```

In [67]:
```
id(b)
```

Out[67]:
```
140719961646280
```

In [68]:
```
id(10)
```
Out[68]:
```
140719961646280
```
In [69]:
```
k = 10
id(k)
```
Out[69]:
```
140719961646280
```
In [70]:
```
a = 20 # as we change the value of the variable then address will also change
id(a)
```
Out[70]:
```
140719961646600
```
In [71]:
```
id(b)
```
Out[71]:
```
140719961646280
```

in python we can't make variable as constant

In [72]:
```
PI = 3.14 #in math this is always constant but in python we can change
PI
```
Out[72]:
```
3.14
```
In [73]:
```
PI = 3.15
PI
```
Out[73]:
```
3.15
```
In [74]:
```
type(PI)
```
Out[74]:
```
float
```

Python Operators

Arithmetic Operator

In [75]:
```
x1, y1 = 10, 5
```
In [76]:
```
x1 + y1
```
Out[76]:
```
15
```
In [77]:
```
x1 - y1
```
Out[77]:
```
5
```
In [78]:
```
x1 * y1
```
Out[78]:
```
50
```
In [79]:
```
x1 / y1
```
Out[79]:
```
2.0
```

In [80]:
```
x1 // y1
```
Out[80]: 2

In [81]:
```
x1 % y1
```
Out[81]: 0

In [82]:
```
x1 ** y1
```
Out[82]: 100000

In [83]:
```
x2 = 3
y2 = 2
x2 ** y2 #power i.e., y2 is the power to x2
```
Out[83]: 9

Assignment Operator

In [84]:
```
x = 2
```

In [85]:
```
x = x + 2 #if you want to increment by 2
```

In [86]:
```
x
```
Out[86]: 4

In [87]:
```
x += 2
x
```
Out[87]: 6

In [88]:
```
x += 2
x
```
Out[88]: 8

In [89]:
```
x *= 2
```

In [90]:
```
x
```
Out[90]: 16

In [91]:
```
x -= 2
x
```
Out[91]: 14

In [92]:
```
x /= 2
x
```
Out[92]: 7.0

In [93]:
```
x //= 2
x
```
Out[93]: 3.0

In [94]:
```
a, b = 5, 6 # you cna assig variable in one line as well
```

In [95]:
```
a
```

Out[95]:
```
5
```

In [96]:
```
b
```

Out[96]:
```
6
```

Unary operator

- unary means 1 || binary means 2
- we apply unary minus operator (-) on the operand n; the value of m become -7, which indicates it as a negative value.

In [97]:
```
n = 7 #negation
n
```

Out[97]:
```
7
```

In [98]:
```
m = -(n)
m
```

Out[98]:
```
-7
```

In [99]:
```
n
```

Out[99]:
```
7
```

In [100…
```
-n
```

Out[100…
```
-7
```

Relation Operator

we are using this operator for comparing

In [104…
```
a = 5
b = 6
```

In [105…
```
a<b
```

Out[105…
```
True
```

In [106…
```
a>b
```

Out[106…
```
False
```

a = b here we can't use = operator that means it is assignig == means equalto

In [109…
```
a == b
```

Out[109…
```
False
```

In [110…
```
a != b # != means not equalto
```

Out[110… True

In [111…
```
# here if I change b=6 to 5
b = 5
```

In [112…
```
a == b
```

Out[112… True

In [113…
```
a
```

Out[113… 5

In [114…
```
b
```

Out[114… 5

In [115…
```
a >= b
```

Out[115… True

In [116…
```
a <= b
```

Out[116… True

In [117…
```
a < b
```

Out[117… False

In [118…
```
a > b
```

Out[118… False

In [119…
```
b = 7
```

In [120…
```
a != b
```

Out[120… True

LOGICAL OPERATOR

- Logical operator is based on the Truth table
- Three important parts of logical operator are --> AND, OR, NOT
- for AND --> 1 & 1 is 1, rest all are 0
- for OR --> 0 | 0 is 0, rest all are 1
- for NOT --> we can reverse the result if we gave True result along with NOT it will give us False

In [121…
```
a = 5
b = 4
```

In [122…
```
a < 8 and b < 5 # refer to Truth table
```

Out[122… True

In [123…
```
a < 8 & b < 2
```

Out[123… False

In [124…

```
a < 8 or b < 2
```

Out[124…

```
True
```

In [125…

```
a > 8 | b < 2
```

Out[125…

```
False
```

In [126…

```
x = False
x
```

Out[126…

```
False
```

In [127…

```
not x #you can reverse the operation
```

Out[127…

```
True
```

In [ ]: